



Instituto Infnet

Quando o dado chega depois

Projeto de Bloco: Inteligência Artificial Aplicada

Aula 10 · Etapa 5 · Páginas dinâmicas com Selenium

Prof. Renan Santos

Onde paramos (Aula 9)

O painel tem seis páginas, uma por pergunta, e o link direto de cada uma funciona.

- **O bilhete: 'página: a pergunta que ela responde'**
Alguém tem duas páginas respondendo à mesma pergunta? Uma delas sobra.
- **A memória comum nasce no roteador; os filtros sobrevivem à troca de página**
É o padrão que a página nova de hoje vai seguir.
- **A promessa da v7: fonte nova = uma linha no menu**
Hoje ela é cobrada.

► Hoje o problema é de COLETA: o dado existe, é público — e não está no HTML.



Dá para trazer o desmatamento?

O pedido da gestora — o dado que todo mundo pergunta.

- **"O INPE publica por estado, todo ano, no painel do PRODES."**
Taxa anual de desmatamento na Amazônia Legal, 9 UFs, de 1988 até hoje — ODS 15.
- **"Tentei copiar a tabela e veio tudo bagunçado. Dá para trazer?"**
Público, licença aberta (CC BY-SA 4.0), robots.txt não proíbe.
- **O reflexo é o da Aula 7: requests + BeautifulSoup**
Não corrija ainda. É o primeiro bloco da aula.

► Na Aula 7 o servidor mandava a página. Hoje ele manda INSTRUÇÕES — e o navegador monta depois.

Onde estamos no Projeto de Bloco

Etapa 5, segunda metade — o item 3 do TP3.



Objetivos da aula

Quatro capacidades — e a primeira é uma decisão.

- **Diagnosticar e decidir**

Página dinâmica? Qual degrau da escada? Por escrito.

- **Pilotar um navegador**

Abrir, ir, localizar, clicar, ler — e FECHAR sempre.

- **Esperar pela condição certa**

E explicar por que as outras esperas falham, até sem erro.

- **Integrar ao projeto**

Coleta à parte, com guarda e snapshot — e saber o preço de pôr o navegador no app.

O item 3 do TP3: 'se necessário'

Leia em voz alta — a frase inteira.

- **"Utilize o Selenium para realizar o web scraping de páginas dinâmicas, SE NECESSÁRIO."**
Não é 'use o Selenium'. É 'decida se precisa'.
- **"... mantendo a simplicidade quando possível."**
Se não precisar: BeautifulSoup ou APIs. E está certo.
- **Armazene em CSV ou TXT, em data/**
O mesmo destino da Aula 7 — e o app só lê.

► O TP3 avalia a DECISÃO. Selenium onde um requests bastava cumpre o item pior, não melhor.



1 · A casca e a escada

A casca

```
python passo01_a_casca.py --online
```

```
B · TerraBrasilis (PRODES) via requests:  
  status 200 · 3098 bytes de HTML  
  texto visível: 'TerraBrasilis' (13 caracteres)  
  tabelas: 0
```

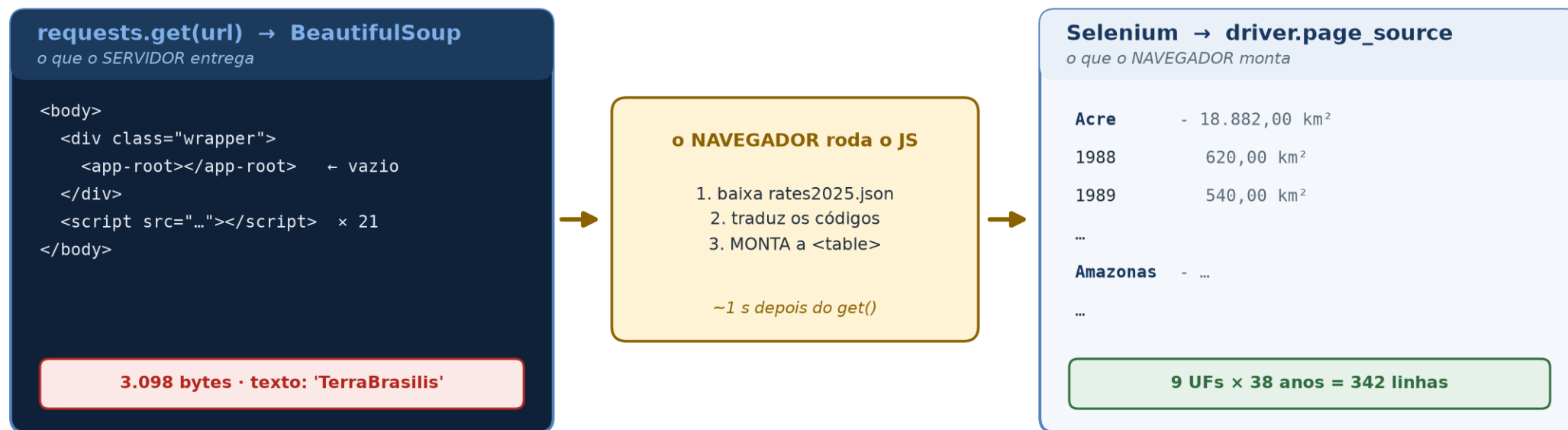
```
# o corpo do HTML:  
#   <div class="wrapper"> <app-root></app-root> </div>  
#   <script src=...></script>   × 21
```

```
# status 200, nenhum erro – e nenhum dado
```

A mesma URL, duas leituras

200 não quer dizer 'veio o dado'. Quer dizer 'veio o que o servidor manda'.

A mesma URL, duas leituras: o que chega × o que o navegador monta



O BeautifulSoup lê o que CHEGOU. Dado que chega depois, por JavaScript, só um navegador vê.

Código-fonte × Inspeccionar — e a aba Rede

Três ferramentas do próprio navegador fazem o diagnóstico antes de qualquer código.

- **Exibir código-fonte (Ctrl+U): o que o SERVIDOR mandou**
É o que o requests recebe. Numa página dinâmica: a casca.
- **Inspeccionar (F12 → Elementos): o DOM DEPOIS do JavaScript**
É o que o Selenium vê. Aqui aparece a tabela.
- **F12 → Rede (Network) → Fetch/XHR → recarregar: DE ONDE o dado veio**
rates2025.json (valores por código de UF) + um 2º arquivo que traduz 18278 → 'Acre'.
Três chamadas de requests — mais rápido que qualquer navegador.

► Antes de pilotar um navegador, olhe a aba Rede. O atalho costuma estar lá.

A escada da coleta

'The use of Selenium is preferred (many times, even as a last option)...' [Chapagain, p195]

A escada da coleta: suba um degrau só quando o de baixo não serve

o que o TP3 diz:

"utilize o Selenium ... se necessário"
"mantendo a simplicidade quando possível"



Selenium é o último degrau, não o primeiro. E cada degrau acima pede um motivo escrito no Data Summary Report.

A decisão da v8 — registrada, e conferida

Aqui o degrau 3 existe. Subimos ao 4 — com motivo escrito.

- **O JSON é INTERNO ao app do INPE**

Sem documentação; o ANO no nome do arquivo (rates2025.json); códigos que pedem um segundo arquivo para virar nome de UF.

- **A tabela renderizada é o que o INPE PUBLICA**

Endereço publicado, nomes e números como aparecem.

- **E uma rota confere a outra**

Passo 5: Selenium × JSON = 342 × 342 pares (UF, ano), ZERO diferenças.

► No SEU projeto, se houver um JSON estável, prefira-o. O que o TP3 cobra é o motivo escrito.

2 · O navegador e as esperas

Abrir, ir, fechar

passo02_o_navegador.py

```
from selenium import webdriver
from selenium.webdriver.common.by import By

opcoes = webdriver.ChromeOptions()
opcoes.add_argument("--headless=new")          # sem janela

with webdriver.Chrome(options=opcoes) as driver:  # o driver vem sozinho
    driver.get(PAGINA)
    linhas = driver.find_elements(By.CSS_SELECTOR, "#tabela tbody tr")
# aqui o navegador já fechou – mesmo se deu erro
```

O que o passo 2 mostra

Três linhas de saída, três lições.

- **'abrir o Chrome levou 2.0 s'**

Segundos, não milissegundos, só para abrir. É um dos motivos para o navegador ficar fora do app.

- **'User-Agent: ... HeadlessChrome/153 ...'**

O robô se anuncia — e não vamos disfarçá-lo. O próprio Chrome avisa que está sendo 'controlled by automated test software' [Chapagain, p200].

- **'logo depois do get(): 0 linha(s) · status: Carregando...'**

get() espera a página CARREGAR. Não espera o JavaScript terminar.

► Entre o get() e o dado existe um intervalo. A aula inteira acontece nele.

Quatro jeitos de esperar

passo03_as_esperas.py

```
# A • sem espera
linhas = driver.find_elements(*LINHAS)
# B • um chute
time.sleep(1); linhas = driver.find_elements(*LINHAS)
# C • implícita – a ÚNICA que o livro mostra [Chapagain, p201]
driver.implicitly_wait(10); linhas = driver.find_elements(*LINHAS)
# D • explícita – pela CONDIÇÃO do dado
WebDriverWait(driver, 10).until(
    EC.text_to_be_present_in_element((By.ID, "status"), "9 UFs carregadas"))
linhas = driver.find_elements(*LINHAS)
```

Os números: 0 · 0 · 1 · 9

Passo 3, medido três vezes — sempre igual.

- **A · sem espera: 0/9 (0,1 s) · B · sleep(1): 0/9 (1,1 s)**

O sleep é um chute: aqui erra por pouco; com 5 s acertaria — e custaria 5 s sempre.

- **C · implícita: 1/9 (1,2 s) — a lista PELA METADE, sem erro nenhum**

Ela faz find_elements voltar assim que existe O PRIMEIRO elemento.

O livro a chama de 'the sleep time' [Chapagain, p201] — e ela não é isso.

- **D · explícita: 9/9 (3,7 s)**

Tenta a cada 0,5 s; volta assim que a condição vale; TimeoutException se não valer.

► B e C ENGANAM. D, se falhar, QUEBRA. Entre um erro que engana e um que quebra, escolha o que quebra.



A tabela vazia que já existia

Tropeço real da v8, no painel do PRODES: a 1ª versão esperava a TABELA aparecer.

- **presence_of_element_located('#tb-area') → terminou em 1,4 s, com 0 linhas**

Medido a cada 50 ms: quando o get() volta (~1,5 s), a <table> já existe — VAZIA.

As 9 UFs e as 342 linhas chegam ~1 s depois, todas de uma vez.

- **Quem salvou a coleta foi a Aula 7**

'NÃO gravei: zero linhas... A coleta anterior foi preservada.' — o CSV ficou byte a byte igual.

- **E a implícita, no PRODES? Funcionou: 342 linhas**

Por sorte: ali as linhas chegam de uma vez. No exercício: 1/9.

► Esperar pelo ELEMENTO não é esperar pelo DADO. 'Funcionou' não é 'está certo'.

A condição certa vem do domínio

```
src/coleta_dinamica.py · esperar_tabela()

# a Amazônia Legal tem 9 UFs – é esse FATO que diz que a tabela está completa
def tabela_completa(d):
    ufs = d.find_elements(By.CSS_SELECTOR, "#tb-area tr.dc-table-group")
    anos = d.find_elements(By.CSS_SELECTOR, "#tb-area tr.dc-table-row")
    return len(ufs) >= 9 and len(anos) > 0

WebDriverWait(driver, 60, poll_frequency=0.25).until(
    tabela_completa, message="a tabela do PRODES não ficou completa a tempo")

# medido: 342 linhas em 2,6 s – sem esperar à toa
```

Livro × hoje — o Selenium também envelheceu

O livro usa Selenium 4.10. O curso, 4.49. Três divergências, ensinadas e não escondidas.

- **Driver baixado à mão, versão pareada ao Chrome [Chapagain, p197–199]**

Hoje o Selenium Manager resolve sozinho: `webdriver.Chrome()` e pronto.

- **Só a espera implícita, descrita como 'the sleep time' [Chapagain, p201]**

`WebDriverWait` não aparece no capítulo. O curso ensina a explícita — e mostra a implícita falhando.

- **'page_source ... can be parsed using ... bs4' [Chapagain, p202] — sem código**

É o caminho do curso: o Selenium roda o JavaScript; o BeautifulSoup extrai.

► Terceira vez no semestre (@st.cache, pages/, e isto): checar a fonte vale para o livro adotado.

A15 · o código (roda sem erro)

coleta_ingenua.py

```
driver = webdriver.Chrome(options=opcoes)
driver.implicitly_wait(10)
driver.get(PAGINA)                # as linhas chegam UMA A UMA
linhas = driver.find_elements(By.CSS_SELECTOR, "#tabela tbody tr")
dados = [tr.text.split() for tr in linhas]

with open("data/processed/tabela.csv", "w", newline="") as f:
    csv.writer(f).writerows(dados)

driver.quit()
```



3 · Interagir, extraire, intégrer



O botão que ainda não estava lá

Passo 4: a página só mostra 2024 depois de um CLIQUE. A 1ª versão esperava as 9 linhas e clicava.

- **ElementNotInteractableException: element not interactable**

A 9ª linha aparece 250 ms ANTES de a página mostrar o botão.

- **Espere pelo que você vai USAR**

```
botao = esperar.until(EC.element_to_be_clickable((By.ID, 'mais')))  
botao.click()
```

- **O clique NÃO devolve o dado**

Ele dispara o JavaScript que vai buscar. Depois dele: esperar de novo (18 linhas).

► O dado antes de ler; o botão clicável antes de clicar; e esperar de novo depois de cada ação.

Selenium roda o JavaScript; BeautifulSoup extrai

src/coleta_dinamica.py

```
driver = abrir_navegador()
try:
    driver.get(URL)
    esperar_tabela(driver)
    html = driver.page_source          # o HTML DEPOIS do JavaScript
finally:
    driver.quit()                     # SEMPRE
ARQ_SNAPSHOT.write_text(html)        # data/raw/: o que o navegador viu

linhas = extrair_tabela(html)         # BeautifulSoup – Aula 7, testável sem navegador
```

Coleta ≠ app — o padrão, e por quê

Não é impossível pôr o navegador no app publicado. É caro — e o padrão do curso é não pôr.

- **O Community Cloud não traz navegador — mas instala o Chromium via packages.txt**

Custa: um Chrome sem janela ocupou 380 MB vazio e ~640 MB com o PRODES (medido aqui);
o app tem de 690 MB a 2,7 GB (documentação do Streamlit, fev/2024).

- **Dois arquivos de requisitos**

requirements.txt (o app; o que o Cloud instala): SEM selenium — o app não o importa.
requirements-coleta.txt: -r requirements.txt + selenium — só na sua máquina.

- **O app só lê o CSV**

Se o site cair ou mudar, o painel segue no ar com a última coleta boa.

► **Padrão: coleta à parte. Navegador no app é possível — com cache e sabendo o preço (passo 6).**

Ética e licença — o que muda com um navegador

Nada do que valia na Aula 7 deixa de valer. Duas coisas se somam.

- **O robots.txt vale para o navegador pilotado**

Ele é um robô. 'The ethical duty of all developers' [Chapagain, p100].

- **Não disfarce o robô**

O livro lista contornar medidas anti-bot entre os usos [p195]. O curso não ensina.

- **CC BY-SA ≠ CC BY**

INPE: citar E compartilhar igual. O CSV que o painel deixa baixar herda a licença.

► **Disfarçar o robô é a linha entre coletar e invadir.**

Anti-padrões do Selenium

Quase todos rodam sem erro.

- **Selenium como primeiro degrau · Selenium no app SEM cache**
Segundos e memória onde um requests bastava; um Chrome novo a cada rerun.
- **time.sleep como espera · espera pela presença do elemento · implícita + find_elements**
Lento sempre; ou 0 linhas; ou lista pela metade — sem erro.
- **Clicar sem esperar o clicável · quit() fora do finally · navegador em cache_resource**
ElementNotInteractableException; navegadores esquecidos; um Chrome aberto para sempre.
- **Disfarçar o robô**
É a linha entre coletar e invadir.

Ao vivo: coletar e publicar a v8

terminal

```
python -m src.coleta_dinamica
```

```
# OK – 342 linhas · 9 UFs · anos 1988–2025 · 7.0 s · navegador
```

```
streamlit run app.py          # a página 🌳 lê o CSV – sem navegador
```

```
git add .
```

```
git commit -m "v8: desmatamento por UF (PRODES/INPE), coletado com Selenium"
```

```
git push                      # o CSV sobe; o Selenium não
```

```
# a fonte nova custou UMA linha no menu – a promessa da v7
```

O TP3 depois de hoje

Três dos cinco itens prontos — e o que falta tem data.

-  **Item 1 — Charter e Data Summary revisados**
Com a Fonte 4 e o MOTIVO do degrau escolhido.
-  **Item 2 — menu e páginas (Aula 9)** ·  **Item 3 — página dinâmica, se necessário**
CSV em data/, coleta à parte.
-  **Item 4 — API com FastAPI (GET e POST)**
Etapa 6 — o TP3 fecha lá.
-  **Item 5 — preparar para LLMs**
Texto das notícias + uma série temporal: o que um LLM resumiria?

► [ASK] No SEU projeto, alguém PRECISA de Selenium? Para a maioria, não — e está certo.

Princípios que levamos

Quatro frases que sobrevivem à próxima versão do Selenium.

- **Suba a escada só o necessário**

E escreva o motivo.

- **Espere pela condição do dado**

E pelo que você vai usar.

- **O navegador fica na coleta — por padrão**

No app, só com cache, com a falha cacheada, e sabendo o preço.

- **Confira uma rota contra a outra**

O jeito mais barato de saber que a extração está certa.



O caso, em 3 lições (v8)

O que a v8 ensinou, na ordem em que doeu.

- **Página dinâmica: o dado chega depois**

requests recebe a casca — e antes do navegador, a aba Rede.

- **Espere pela condição, não pela presença**

A <table> existia vazia; a guarda da Aula 7 salvou a coleta.

- **O navegador fica na coleta — por padrão**

O app pagou a fonte nova com uma linha no menu. Levar o navegador junto tem preço.

► **Etapas 6: e se OUTRO PROGRAMA quiser esses dados? Uma API nossa, com FastAPI.**